PRD

RAG-Based Document Management System — AI-Powered Digital Repository Dashboard

Field	Details
Version	1.0
Date	March 1, 2026
Status	Final
Prepared For	Indore Institute of Science & Technology, Indore
Project Owner	Vibhav Jain
Author	Nishita
Co-Author	Udit Rathod, Rishi Singh, Rudra Dewatwal
Reviewers	IT Department & Academic Department
Approval	Project Guide - Vibhav Jain
Next Review Date	•

Table of Contents

1. Executive Summary
2. Product Objectives & Success Metrics
3. User Personas
4. Functional Requirements
5. Technical Requirements
6. Non-Functional Requirements
7. User Experience (UX) Requirements
8. Implementation Roadmap
9. Risk Assessment & Mitigation
10. Budget Estimate
11. Appendix

---

1. **Executive Summary**

1.1 Vision

To transform institutional document management by building an AI-powered digital repository that eliminates paper-based inefficiencies and enables instant, intelligent access to circulars, syllabi, question papers, and certificates. By leveraging RAG-based search powered by LangChain, Ollama, and Pinecone, the system empowers students, faculty, and administrators to retrieve the right document at the right time — through a smart kiosk or dashboard — with full version control and zero manual effort.

1.2 Proposed Solution

Current institutional document management processes suffer from three critical pain points:

Pain Point	Description
No Central Repository	Documents are scattered across departments with no unified digital storage system
Manual Retrieval Delays	Students and staff spend 10–15 minutes manually locating documents from physical files
No Version Tracking	Updated circulars and syllabi overwrite old versions with no history or rollback capability

1.3 Solution Overview

The RAG-Based Document Management System addresses these pain points through three core capabilities:

Solution	Description
AI-Powered Digital Repository	A centralized platform that digitizes and stores all institutional documents — circulars, syllabi, question papers, and certificates — in one searchable location
RAG-Based Semantic Search	Natural language search powered by LangChain, Ollama, and Pinecone delivers accurate document retrieval in under 3 seconds, eliminating manual searching entirely
Version Control System	Every document update is automatically versioned with full history, timestamps, and one-click rollback — ensuring the right version is always accessible

1.4 Expected Impact

Metric	Target
Document Retrieval Time	Reduced from 10–15 minutes to under 3 seconds
Search Accuracy	Greater than 85% on benchmark queries
Documents Indexed	10,000+ institutional documents
Manual Effort Reduction	80% reduction in staff time spent on document handling
System Uptime	99% availability for uninterrupted access
User Satisfaction	Greater than 4 out of 5 rating from students and faculty
Version Tracking Coverage	100% of documents with full version history
Concurrent Users Supported	50+ simultaneous users on kiosk and dashboard

---

2. Product Objectives & Success Metrics

2.1 Primary Objectives

Objective	KPI	Target	Measurement Method
Build a centralized digital repository for all institutional documents	Total documents indexed	10,000+ documents	Database record count
Enable AI-powered semantic search using RAG pipeline	Search accuracy & response time	>85% accuracy, <3 seconds	Benchmark query testing
Implement version control for all document updates	Version history coverage	100% of documents tracked	System version log audit
Provide self-service terminal for students and staff	Student self-service rate	80% queries resolved without staff help	Kiosk session analytics
Deliver admin dashboard with analytics and user management	Dashboard adoption rate	90% of admins using dashboard weekly	Login and activity logs

Ensure data privacy with fully local LLM processing	Data leakage incidents	Zero external API calls for LLM	Network traffic monitoring
Achieve high system availability and performance	System uptime	99% availability	Server monitoring tools

2.2 Secondary Objectives

- Scalability: Handle 10,000+ documents and support 50+ concurrent users during peak academic seasons
- OCR Support: Process and extract text from scanned circulars, certificates, and question papers with greater than 90% accuracy
- Version Control: Maintain complete document history with timestamps, author tracking, and one-click rollback capability
- Data Privacy: Ensure 100% local LLM processing via Ollama with zero data sent to external APIs
- Role-Based Access: Enforce strict access control across Admin, Faculty, and Student user roles with zero unauthorized access incidents
- Analytics & Reporting: Provide real-time document usage analytics with one-click export to PDF and Excel formats

---

3. User Personas

3.1 Primary User — Student

Attribute	Detail
Role	Undergraduate / Postgraduate Student
Primary Goal	Quickly access syllabus, circulars, question papers, and certificates
Pain Point	Manual search in department office takes 10–15 minutes
Access Method	Self-Service Kiosk / Dashboard
Usage Frequency	3–5 times per semester

Key Actions:

- Search using natural language
- View document preview
- Download or print document
- Access document via QR code

Success Criteria:

Document retrieved in **under 3 minutes** without staff assistance.

3.2 Secondary User — Faculty

Attribute	Detail
Role	Department Faculty Member
Primary Goal	Upload, update, and manage academic documents
Pain Point	No tracking of document versions; old files get overwritten
Access Method	Admin Dashboard
Usage Frequency	2–3 times per week

Key Actions:

- Upload new/updated documents
- System auto-creates new version
- View version history
- Rollback incorrect uploads
- Search documents using AI

Success Criteria:

Document upload and versioning completed in **under 5 minutes**.

3.3 Tertiary User — Admin

Attribute	Detail
Role	System Administrator
Primary Goal	Manage users, monitor system, ensure compliance
Key Actions	Approve uploads, manage roles, view analytics, audit logs
Success Criteria	Maintain 99% uptime and zero unauthorized access

---

4. Functional Requirements

4.1 Document Ingestion & Management

- Accept PDF, DOCX, JPG, and PNG document formats for upload
- Perform OCR processing to extract text from scanned documents using Tesseract
- Auto-categorize documents as Circular, Syllabus, Question Paper, or Certificate
- Capture metadata including department, subject, year, date, and uploaded by
- Support bulk upload of multiple documents simultaneously
- Allow document preview before downloading or printing

4.2 AI-Powered Search (RAG Pipeline)

- Enable users to search documents using plain English natural language queries
- Embed documents and store vectors in Pinecone vector database
- LangChain agent retrieves the most relevant document chunks based on query
- Ollama LLM generates accurate answers with source document citations
- Filter search results by category, year, department, and date
- Allow users to view and repeat recent search history

4.3 RAG Workflow

Flow 1: Document Search

Student: "Show me the 2024 Computer Science question paper."

↓

[Query Received] → [Convert to Vector Embedding] → [Pinecone Semantic Search]

↓

[Retrieve Relevant Document Chunks]

↓

[Ollama LLM Generates Answer]

↓

[Display Document with Preview & Download]

Flow 2: Filtered Search

Student: "Show me all IT department circulars from 2023."

↓

[Query Received] → [Apply Filters: Department = IT, Year = 2023] → [Pinecone Search]

↓

[Filtered Document Chunks Retrieved]

↓

[Ollama LLM Summarises Results]

↓

[Display Filtered Document List with Preview]

Flow 3: Document Not Found

Student: "Show me the 2020 MBA syllabus."

↓

[Query Received] → [Convert to Vector Embedding] → [Pinecone Search]

↓

[Low Similarity Score Detected — No Exact Match]

↓

[Retrieve 3 Closest Related Documents]

↓

[Display Results with Suggestion to Refine Search Query]

4.1.3 Human Handoff Protocol

Trigger Conditions for Human Handoff:

- Document not found after 2 search attempts
- The user requests to speak to admin directly
- System confidence score below threshold
- A critical document urgently required

4.3.1 Dashboard Components

A. Real-Time Document Tracking

- Live document upload and search activity counter
- Active user sessions on terminal and dashboard
- Most searched documents in real-time

B. Document Search Funnel

Query Entered → Embedding Generated → Pinecone Search → Chunks Retrieved → LLM Answer Generated → Document Accessed

↓ ↓ ↓ ↓ ↓ ↓

Total Queries Vector Match % Relevance Score Chunks Found Answer Quality Download / Print

C. Source-Wise ROI Analysis

Source	Total Documents	Total Searches	Avg Response Time	Search Success Rate	User Satisfaction	ROI
Circulars	X	Y	Z sec	A%	B/5	Cx
Syllabus	X	Y	Z sec	A%	B/5	Cx
Question Papers	X	Y	Z sec	A%	B/5	Cx
Certificates	X	Y	Z sec	A%	B/5	Cx
Scanned Docs (OCR)	X	Y	Z sec	A%	B/5	Cx

D. Counselor Performance Scorecard

Metric	Weight	Calculation
Document Upload Rate	20%	Avg. number of documents uploaded per week by faculty
Search Resolution Rate	25%	% of user searches returning accurate and relevant results
Version Control Compliance	30%	% of updated documents with proper version history maintained
User Satisfaction Score	15%	Post-session star rating collected from students at terminal
Document Expiry Management	10%	% of expired circulars and syllabi archived within 7 days

4.2.2 predictive analysis

—

---

5. Technical Requirements

5.1 Architecture Overview

```
┌──────────────────────────────────────────┐
│ 1️⃣ USER LAYER
├──────────────────────────────────────────┤
• Student — Self-Service Kiosk
• Faculty — Admin Dashboard
• Admin — Admin Dashboard
```

↓

```
┌──────────────────────────────────────────┐
│ 2️⃣ FRONTEND LAYER
├──────────────────────────────────────────┤
• React.js Dashboard
```

```
• Touch-Friendly Kiosk Interface
• Chat-Style AI Search UI
```

↓

```
────────────────────────────────────────
3️⃣ API LAYER
────────────────────────────────────────
• FastAPI Backend
• REST APIs
• JWT Authentication
• Role-Based Access Control (RBAC)
```

↓

```
────────────────────────────────────────
4️⃣ PROCESSING LAYER
────────────────────────────────────────
```

```
AI Orchestration
• LangChain (RAG Pipeline Manager)
```

```
OCR Processing
• Tesseract OCR
• PyMuPDF (PDF Text Extraction)
```

```
Version Control
• Auto-Versioning Engine
• Version History Log
• Rollback Manager
```

↓

```
────────────────────────────────────────
5️⃣ INTELLIGENCE LAYER
────────────────────────────────────────
• Ollama (Local LLM Runtime)
• Llama 3 / Mistral (LLM Model)
• SentenceTransformers (Embedding Model)
```

↓

```
────────────────────────────────────────
6️⃣ STORAGE LAYER
────────────────────────────────────────
• FAISS (Vector Database)
```

```
• PostgreSQL (Metadata + Users + Roles)
• Local Storage / AWS S3 (Document Files)
```

5.1.1 Layer Description

Layer	Component	Purpose
User Layer	Terminal, Dashboard	End user interaction points
Frontend Layer	Streamlit / React	UI for kiosk and dashboard
API Layer	FastAPI + RBAC	Request handling and authentication
AI Layer	LangChain Agent	RAG pipeline orchestration
OCR Layer	Tesseract / PyMuPDF	Scanned document text extraction
Version Control	Auto-Versioning Engine	Document version tracking and rollback
Intelligence Layer	Ollama LLM + Embeddings	Answer generation and vectorization
Storage Layer	Pinecone + PostgreSQL + S3	Vector, metadata, and file storage

5.2 Technology Stack

Layer	Technology
Frontend	React.js + Tailwind CSS
Backend	FastAPI (Python) + Uvicorn
Authentication	JWT + Bcrypt
Database	PostgreSQL
Vector Database	FAISS
LLM Runtime	Ollama
LLM Model	Llama 3 / Mistral
Embeddings	SentenceTransformers (all-MiniLM-L6-v2)
OCR	Tesseract OCR
PDF Processing	PyMuPDF
Deployment	Local Server / Docker
Version Control	Git

5.3 Integration Requirements

1. **Mandatory Integrations:**

- Ollama (Local LLM Runtime) — for AI answer generation within the RAG pipeline
- Large Language Model (Llama 3 / Mistral) — for contextual natural language response generation
- FAISS (Vector Database) — for semantic similarity search and document retrieval
- Relational Database (PostgreSQL / MySQL) — for storing users, roles, document metadata, and version history
- SentenceTransformers (Embedding Model) — for converting document text and user queries into vector embeddings
- PDF Processing Library (pdfplumber / PyPDF) — for extracting text from uploaded PDF documents
- JWT Authentication System — for secure login and role-based access control
- FastAPI REST APIs — for frontend-backend communication

Optional Integrations:

- **OCR Engine (Tesseract OCR)**—for extracting text from scanned PDF documents
- **Redis (Caching Layer)**—for improving response time of frequently asked queries
- **Docker**—for containerized deployment and easier scalability
- **Nginx**—for reverse proxy and production-level performance optimization
- **Prometheus/Grafana—for** system monitoring and performance tracking
- **Cloud Hosting (AWS/Azure)—for** scalable remote deployment
- **Email/Notification Service (SMTP)**—for document upload or update alerts
- **Logging System (ELK Stack)**—for centralized logging and debugging

5.4 Security & Compliance

Control Area	Implementation
Authentication	JWT-based secure login with token expiry
Authorization	Role-Based Access Control (Admin, Faculty, Student)
Password Security	Bcrypt hashing
API Security	HTTPS + Input validation
File Security	PDF-only uploads, max 20MB
Data Privacy	Fully local LLM via Ollama (No external API calls)
Audit Logs	Track logins, uploads, updates, searches
Database Security	Parameterized queries + ORM
Backup Policy	Daily automated backup (DB + FAISS index)
Session Control	Auto logout after inactivity

---

6. Non-Functional Requirements

6.1 Performance Metrics & Requirements

Metric	Target
Query Response Time	≤ 5 seconds
Upload Processing Time	≤ 20 sec (10MB file)
Retrieval Accuracy	≥ 85%
Concurrent Users	50+ users
System Uptime	99%
Vector Search Speed	≤ 1 sec (10k chunks)
Metadata Query Speed	≤ 2 sec
Max Upload Size	20MB
Error Recovery	≤ 2 minutes

6.2 Scalability Requirements

- The system shall support **1,000+ uploaded documents** initially and be scalable to 10,000+ without architectural redesign.
- The system shall handle **50,000+ document chunks** in the vector database without significant performance degradation.
- The backend shall support scaling from **50 to 200 concurrent users** with server configuration upgrades.
- The architecture shall allow **horizontal scaling** through containerization (e.g., Docker-based deployment).
- The system shall allow **LLM upgrades or model replacement** without major codebase changes.

6.3 Reliability Requirements

- The system shall maintain **99% uptime** during institutional operational hours.
- The system shall implement **automatic error handling** to prevent crashes during document processing or query execution.
- The system shall ensure **daily backups** of database and vector index files.
- The system shall maintain **data integrity**, preventing loss of document metadata or version history.
- The system shall provide **graceful failure responses** without exposing internal server details to users.

---

7. User Experience (UX) Requirements

7.1 AI Assistant Personality

Attribute	Detail
Tone	Friendly, intelligent, professional yet approachable academic assistant
Name	TBD — e.g., “DocuAssist” or “CampusAI”
Avatar	Professional academic-themed illustration (minimal, clean design)

> **Sample Interactions:**

> **Greeting:**

> “Hello 👋 I’m your Academic Document Assistant. I can help you find exam dates, syllabus details, circulars, or notices. What would you like to know?”

> **Uncertainty:**

> “I’m not fully confident about this information. Let me retrieve the most relevant document references to ensure accuracy.”

> **Source Confirmation:**

> “I found this information in the ‘Semester Examination Circular—March 2026.' Would you like to open the full document?”

> **Escalation (Future Enhancement):**

> “This query may require administrative clarification. Please contact the academic office for further confirmation.”

7.2 Dashboard UX

Design Principles:

- Clean, minimal dashboard interface
- Mobile-responsive layout
- Dark mode support for extended reading
- Simple document upload (drag-and-drop support)
- Clear separation between the search and Chat interface
- Visible document source references for transparency

Key Interactions:

- Upload and categorize academic documents in 2–3 steps
- Search documents using keywords or natural language
- Chat-style AI interaction with contextual responses
- Clickable source references linking to exact PDF
- Filter documents by department, category, and date
- View document version history

---

8. Implementation Roadmap

Phase 1 — Foundation (Weeks 1–2)

☐ Finalize system architecture and database schema

☐ Backend setup using FastAPI

☐ User authentication (JWT) and role-based access control

☐ Document upload functionality (PDF only)

☐ Metadata storage (department, category, versioning)

☐ Basic frontend setup (React dashboard structure)

☐ Initial deployment on local server

Phase 2 — Intelligence (Weeks 3–4)

☐ PDF text extraction implementation

☐ Text chunking mechanism development

☐ Embedding model integration (SentenceTransformers)

☐ FAISS vector database setup and indexing

☐ Query embedding generation

☐ Similarity search (top-k retrieval)

☐ Ollama integration with LLM (Llama 3 / Mistral)

☐ Context-based AI answer generation with source reference display

Phase 3 — Optimization & Integration (Weeks 5–6)

☐ Chat-style frontend integration

☐ Document filtering (department, category, date)

☐ Version history tracking system

☐ Performance optimization (chunk tuning, indexing improvements)

☐ Response time optimization (<5 sec target)

☐ Error handling and logging implementation

☐ System testing (functional + performance testing)

Phase 4 — Enhancement & Deployment (Weeks 7–8)

☐ Admin dashboard improvements

☐ Audit logs and monitoring integration

☐ Backup and recovery implementation

☐ Docker containerization (optional)

☐ Final deployment configuration

☐ Documentation completion

☐ Demo preparation and presentation material

---

9. Risk Assessment & Mitigation

Risk	Probability	Impact	Mitigation Strategy
LLM Hallucination (Incorrect AI responses)	Medium	High	Use strict context injection from retrieved chunks and always display source references.
Poor Retrieval Accuracy	Medium	High	Optimize chunk size, improve embedding model selection, and fine-tune top-k retrieval settings.
Slow Query Response Time	Medium	Medium	Optimize FAISS indexing, limit chunk retrieval count, and use async backend processing.
Large Document Processing Delay	Medium	Medium	Implement background processing for embedding generation and limit upload file size.
Data Loss (Database or FAISS Index)	Low	High	Implement daily automated backups and version control for documents.
Unauthorized Access / Security Breach	Low	High	Implement JWT authentication, role-based access control, and encrypted password storage.
Server Downtime	Medium	Medium	Deploy on stable infrastructure with monitoring and restart policies.
Scalability Limitations	Medium	Medium	Use modular architecture and allow future horizontal scaling (Docker/containerization).
Model Compatibility Issues (Ollama updates)	Low	Medium	Maintain version control of LLM models and test updates in staging environment.
User Misinterpretation of AI Responses	Medium	Medium	Clearly display document source, date, and context snippet with every AI answer.

10. Appendix

10.1 Glossary

Term	Definition
RAG (Retrieval-Augmented Generation)	AI architecture that retrieves relevant documents before generating an answer using an LLM.
LLM (Large Language Model)	AI model trained on large datasets to understand and generate human-like text.
LangChain	Framework used to orchestrate LLM workflows and build RAG pipelines.
Ollama	Local runtime environment for deploying and running LLMs offline.
FAISS	Vector similarity search library used for efficient document retrieval.
Embeddings	Numerical vector representations of text used for semantic search.
SentenceTransformers	Model used to generate embeddings for documents and queries.
OCR (Optical Character Recognition)	Technology used to extract text from scanned documents or images.
JWT (JSON Web Token)	Secure authentication mechanism used for user login and authorization.
RBAC (Role-Based Access Control)	Access control mechanism based on user roles (Admin, Faculty, Student).
Metadata	Structured information describing a document (department, year, category, etc.).
Vector Database	Database optimized for storing and searching embedding vectors.
Version Control	System that tracks document updates and maintains historical versions.
Chunking	Process of dividing documents into smaller segments for better retrieval accuracy.
Top-K Retrieval	Method of retrieving the top most relevant document chunks based on similarity score.

10.2 References

- LangChain Official Documentation – https://docs.langchain.com
- Ollama Official Documentation – https://ollama.com
- Meta AI – Llama 3 Model Documentation
- Mistral AI – Mistral Model Documentation
- FAISS – Facebook AI Similarity Search Documentation
- PostgreSQL Global Development Group – PostgreSQL Documentation
- FastAPI Official Documentation
- Tesseract OCR Documentation
- Hugging Face – SentenceTransformers Documentation
- Academic Research Papers on Retrieval-Augmented Generation (2020–2024)

10.3 Success Stories

- **Digital Repository Adoption:** Institutions implementing centralized document systems reduced manual document handling by up to 80% and improved retrieval efficiency.
- **AI-Based Semantic Search:** Universities using RAG-based search reduced document retrieval time from 10–15 minutes to under 5 seconds.
- **Local LLM Deployment:** Organizations deploying LLMs locally ensured data privacy, zero external API dependency, and compliance with institutional policies.
- **Version Control Implementation:** Automated document version tracking improved transparency and eliminated confusion caused by outdated circulars and syllabi.